# Matrix factorisation approach to movie recommendation

**Toavina Thierry Leong-Pock-Tsy** [1]

## Abstract

Recommendation systems are essential components of modern online platforms, providing personalised user experiences by suggesting items users are likely to enjoy. Matrix factorisation stands as one of the most successful techniques for building recommendation systems. However, deploying these methods in large-scale, real-world settings presents significant challenges due to the nature of the data and the amount of computation needed. This paper explores several matrix factorisation variants for movie recommendation, examining their theoretical foundations and the practical challenges that arise during implementation and deployment.

The code is available on github: https://github.com/Toavina00/Recommender-system

## 1. Introduction

Modern online platforms rely heavily on recommender systems to personalise content discovery and enhance user engagement.

When building recommender systems, two main approaches are typically considered: *content filtering* and *collaborative filtering*. Content filtering uses descriptive information about users and items to characterise them, then matches users with items based on these profiles. In contrast, collaborative filtering leverages user–item interaction data, matching users not only with their own past preferences but also with the preferences of similar users. Interactions can be divided into two categories: explicit feedback, such as ratings or purchases, and implicit feedback, such as clicks or viewing time. (Koren et al., 2009)

[1]African Institute for Mathematical Sciences (AIMS) South Africa, 6 Melrose Road, Muizenberg 7975, Cape Town, South Africa. Correspondence to: Toavina Thierry Leong-Pock-Tsy <toavina@aims.ac.za>.


Collaborative filtering has proven highly effective by learning to match users and items from interaction patterns. Since interaction data is often more readily available than detailed item or user metadata, these methods are generally favoured in practice over purely content-based approaches. However, applying these methods to large-scale datasets introduces several challenges. Interaction data typically follows a long-tail distribution, meaning many items receive few ratings, which complicates modelling and evaluation. Additionally, computational efficiency becomes critical at scale. A further limitation is the *cold start* problem (Koren et al., 2009), where the model performs poorly for new users or items with few interactions.

In this paper, we conduct a systematic study of matrix factorisation variants for movie recommendation, addressing both theoretical foundations and practical implementation concerns.

## 2. Related Work

The most successful collaborative filtering techniques are models based on matrix factorisation, which aim to represent the users and items in a latent space. Each user and item are represented using vectors in this latent space, and their interactions are defined as a function of those vectors (Koren et al., 2009). In our movie recommendation task, we approximate the rating with the dot product of the latent vectors of the users and movies.

In real-world data, the distribution of the items bought in a marketplace typically follows a power-law distribution. This is evident from the fact that highly popular items are purchased exponentially more frequently than those that are less popular. Because of this distribution, most of the items, being at the tail of the distribution, are often overlooked during recommendations. In (Koenigstein et al., 2012), they have explored variational matrix factorisation to tackle this issue, and we will later see this technique when applied to movie recommendation.

External information about the items is often unavailable, which is one of the reasons that makes collaborative filtering preferable to content filtering techniques. However, in the case where it is available, it could be potentially useful, especially when dealing with the cold start problem. In

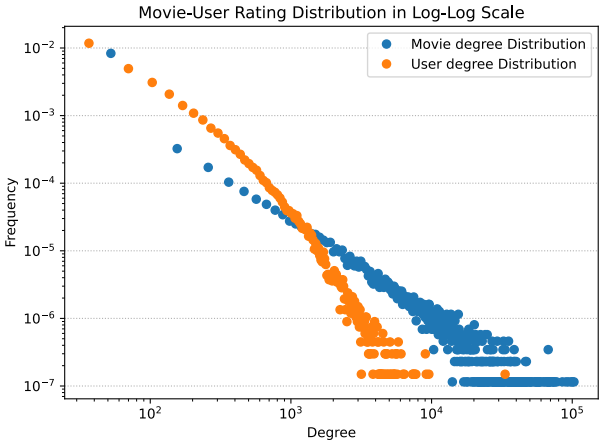


*Figure 1.* Movie and User degree distributions

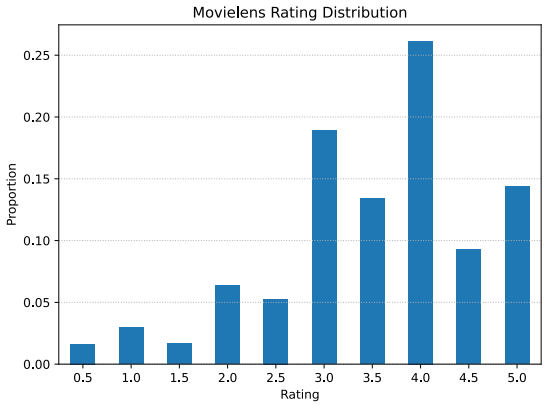


*Figure 2.* Rating distributions

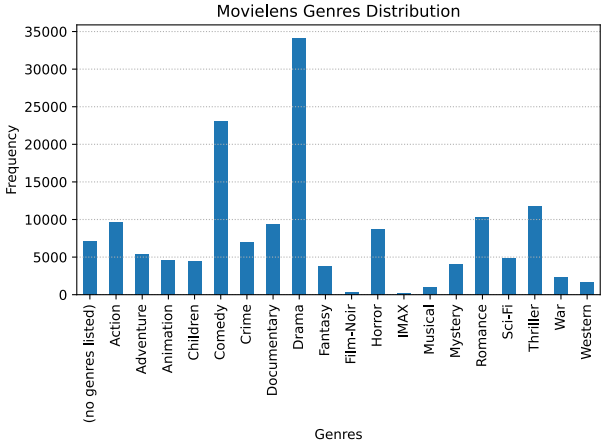


*Figure 3.* Movie genres distribution

(Koenigstein & Paquet, 2013), they proposed an approach to incorporate features in the collaborative filtering system, and we will later see an approach inspired by their work that incorporates movie genres into the model.

## 3. Dataset

The model is trained using the MovieLens dataset (Harper & Konstan, 2015) that contains 32,000,204 ratings across 87,585 movies and 200,948 users that have rated at least 20 movies (GroupLens). The distribution of ratings per user and per movie, in other words, the degree distribution, follows a power law. This distribution is governed by the equation $p_k \sim k^{-\gamma}$ with the exponent $\gamma$ being its degree exponent. (Barabási & Pósfai, 2016). In log-log scale we have $log(p_k) \sim -\gamma log(k)$ so $log(p_k)$ depends linearly on $log(k)$ with slope $-\gamma$. We can observe the degree distribution in log-log scale in Figure 1. We can see that in this scale the movie degree distribution is approximately linear, confirming the power-law behaviour. The user degree distribution, on the other hand, is only approximately linear up to a certain degree; it is mostly due to the fact that our dataset contains only the users that have rated at least 20 movies, and if given the full dataset, the power law is expected to be more apparent.

In our dataset, each movie rating ranges from $0.5$ to $5.0$ incrementing by $0.5$, and we can observe the rating distribution in Figure 2. We see that the majority of movies have ratings between $3.0$ and $4.0$.

Additionally, each movie is labelled with one or many genres across $19$ genres. In the case where they are not considered to belong in any of the $19$ genres, they are labelled as *(no genres listed)*. We can see that most movies fall under *Comedy* or *Drama* and only a few are classified are *Film-Noir* or *IMAX*. We can see this in Figure 3.

## 4. Model

For any user $m$ and movie $n$, we define $u_m \in \mathbb{R}^K$ and $v_n \in \mathbb{R}^K$ their respective latent vector representations or embeddings, and $b_m^{(u)} \in \mathbb{R}$ and $b_n^{(v)} \in \mathbb{R}$ their respective bias. We define $U = \{u_m\}_{m=1}^M$, $V = \{v_n\}_{n=1}^N$, $b^{(u)} = \{b_m^{(u)}\}_{m=1}^M$ and $b^{(v)} = \{b_n^{(v)}\}_{n=1}^N$ the set of all embeddings and biases for the users and movies. We denote $r_{mn}$ the rating given by a user $m$ to a movie $n$ and $R = \{r_{mn}\}_{(m,n)}$ the set of all ratings for each pair of user and movie. We then make the following assumptions for our model:

$$r_{mn}|u_m, v_n, b_m^{(u)}, b_n^{(v)} \sim N(r_{mn}; u_m^T v_n + b_m^{(u)} + b_n^{(v)}, \lambda^{-1})$$
$$u_m \sim N(u_m; 0, \tau^{-1})$$
$$v_n \sim N(v_n; 0, \tau^{-1})$$
$$b_m^{(u)} \sim N(b_m^{(u)}; 0, \gamma^{-1})$$

$$b_n^{(v)} \sim N(b_n^{(v)}; 0, \gamma^{-1})$$

Then we try to find the parameters $u_m, v_n, b_m^{(u)}, b_n^{(v)}$ that maximizes the posterior probability

$$p(u_m, v_n, b_m^{(u)}, b_n^{(v)} | r_{mnn})$$

We can derive the expression of this posterior using Bayes' theorem.

$$p(U, V, b^{(u)}, b^{(v)}|R) = \frac{p(R|U, V, b^{(u)}, b^{(v)})p(U, V, b^{(u)}, b^{(v)})}{p(R)}$$

It is more convenient to define our loss as the negative logarithm of this posterior. We will then minimise the following cost instead:

$$\begin{aligned} Cost &= -log(P(U, V, b^{(u)}, b^{(v)}|R) \\ &= -log(P(R|U, V, b^{(u)}, b^{(v)})P(U, V, b^{(u)}, b^{(v)})) \\ &\quad + P(R) \\ Cost &= \frac{\lambda}{2} \sum_{m=1}^{M} \sum_{n \in \Omega(m)} (r_{mn} + u_m^T v_n - b_m^{(u)} + b_n^{(v)})^2 \\ &\quad + \frac{\tau}{2} \sum_{m=1}^{M} u_m^T u_m + \frac{\tau}{2} \sum_{n=1}^{N} v_n^T v_n \\ &\quad + \frac{\gamma}{2} \sum_{m=1}^{M} (b_m^{(u)})^2 + \frac{\gamma}{2} \sum_{n=1}^{N} (b_n^{(v)})^2 + C \end{aligned}$$

with $\Omega(m)$ defined as being the set of all movies that the user $m$ has rated.

### 4.1. Alternating Least Square

We minimise our cost function using the alternating least squares algorithm, which alternatively finds the optimal user and movie biases and embeddings. We update a parameter by fixing all the others according to the derivative of the cost in regard to that parameter. The implementation is shown in Algorithm 1.

The update rule for $b_m^{(u)}$ is expressed as follows:

$$\frac{dCost}{db_m^{(u)}} = 0, \quad b_m^{(u)} = \frac{\lambda \sum_{n \in \Omega(m)} (r_{mn} - u_m^T v_n - b_n^{(v)})}{\lambda |\Omega(m)| + \gamma}$$

The update rule for $u_m$ is expressed as follows:

$$\frac{dCost}{du_m} = 0, \quad u_m = M(\lambda \sum_{n \in \Omega(m)} (r_{mn} - b_m^{(u)} - b_n^{(v)}) v_n)$$

with,

**Algorithm 1** Alternating Least Square

**Input:** $D = \{(\text{user}, \text{movie}, \text{rating})\}$, $\mathbf{U}[M, K]$ (User embedding matrix), $\mathbf{b}_u[M]$ (User bias vector), $\mathbf{V}[N, K]$ (Movie embedding matrix), $\mathbf{b}_m[N]$ (Movie bias vector), $I$ (number of iteration)
**for** iter $i = 0$ **to** $I$ **do**
  **for** user $m = 1$ **to** $M$ **do**
    Update $\mathbf{b}_u[m]$ using (movie, rating) from the user $m$ ratings **in** $D$
    Update $\mathbf{U}[m, :]$ using (movie, rating) from the user $m$ ratings **in** $D$
  **end for**
  **for** movie $n = 1$ **to** $N$ **do**
    Update $\mathbf{b}_v[n]$ using (user, rating) from the movie $n$ ratings **in** $D$
    Update $\mathbf{U}[n, :]$ using (user, rating) from the movie $n$ ratings **in** $D$
  **end for**
**end for**

$$M = (\lambda \sum_{n \in \Omega(m)} v_n v_n^T + \tau I)^{-1}$$

See the full derivatives in the appendix. By symmetry of the problem, we can derive similar expressions for both movie bias and embeddings.

### 4.2. Mean field variational inference

To better deal with rare movies, we make predictions by averaging over all plausible settings of $U, V, b^{(u)}, b^{(v)}$. Here we will denote $R$ to be the set of all ratings in our dataset. For any user $i$ and movie $j$, the predictive distribution can be expressed as follows:

$$\begin{aligned} p(r_{ij}|R) = \int & p(r_{ij}|u_i, v_j, b_i^{(u)}, b_j^{(v)}, R) \\ & \times p(u_i, v_j, b_i^{(u)}, b_j^{(v)}|R) d\{u_i, v_j, b_i^{(u)}, b_j^{(v)}\} \end{aligned}$$

Here we still model the conditional rating as before. This integral is analytically intractable, so in (Koenigstein et al., 2012) they proposed factorising the posterior distribution to approximate the predictive distribution.

$$p(u_i, v_j, b_i^{(u)}, b_j^{(v)}|R) = q(u_i)q(v_j)q(b_i^{(u)})q(b_j^{(u)})$$

We can then approximate the integral as,

$$\begin{aligned} p(r_{ij}|R) = \int & p(r_{ij}|u_i, v_j, b_i^{(u)}, b_j^{(v)}, R) \\ & \times q(u_i)q(v_j)q(b_i^{(u)})q(b_j^{(u)}) d\{u_i, v_j, b_i^{(u)}, b_j^{(v)}\} \end{aligned}$$

In (Bishop, 2006), the author describes a way to find the factorisation. They state the problem as follows: For a given set of latent variables $Z$ and observed variables $X$, we want to find $q$ such that

$$q(Z) = \prod_i q(Z_i) \approx p(Z|X)$$

The KL divergence between the factorisation $q$ and the true posterior $p$ is expressed as,

$$KL(q||p) = -\int q(Z) log\Big(\frac{p(Z|X)}{q(Z)}\Big) dZ$$

The goal is to find $q$ that which minimises the above expression. However, it is more convenient to maximise the following expression instead:

$$L[q] = log(p(X)) - KL(q||p)$$

$$L[q] = \int q(Z) log\Big(\frac{p(X,Z)}{q(Z)}\Big) dZ$$

This would still give an optimal $q$ as $log(p(X))$ is constant; hence, maximising $L[q]$, also called Evidence lower bound or ELBO, is equivalent to minimising $KL(q||p)$.

In our model, we want to find a full factorisation of the parameters. And we can show that this factorisation has the following form:

$$\begin{aligned}
q(U) &= \prod_{m=1}^{M} q(u_m) = \prod_{m=1}^{M} \mathcal{N}(u_m; \mu_m^{(u)}, \Sigma_m^{(u)}) \\
&\approx \prod_{m=1}^{M} \prod_{k=1}^{K} \mathcal{N}(u_{mk}; \mu_{mk}^{(u)}, {\sigma_{mk}^{(u)}}^2) \\
q(V) &= \prod_{n=1}^{N} q(v_n) = \prod_{n=1}^{N} \mathcal{N}(v_m; \mu_n^{(v)}, \Sigma_n^{(v)}) \\
&\approx \prod_{n=1}^{N} \prod_{k=1}^{K} \mathcal{N}(v_{nk}; \mu_{nk}^{(v)}, {\sigma_{nk}^{(v)}}^2) \\
q(b^{(u)}) &= \prod_{m=1}^{M} q(b_m^{(u)}) = \prod_{m=1}^{M} \mathcal{N}(b_m^{(u)}; \mu_m^{(bu)}, {\sigma_m^{(bu)}}^2) \\
q(b^{(v)}) &= \prod_{n=1}^{N} q(b_n^{(v)}) = \prod_{n=1}^{N} \mathcal{N}(b_n^{(v)}; \mu_n^{(bv)}, {\sigma_n^{(bv)}}^2)
\end{aligned}$$

And the expression $L[q]$ we want to maximise is the following:

$$L[q] = E_q\Big[log\Big(\frac{p(R,U,V,b^{(u)},b^{(v)})}{q(U)q(V)q(b^{(u)})q(b^{(v)})}\Big)\Big]$$

We train the model by iteratively updating the parameters of the factorised distribution, similar to alternating least squares. For each factor, we keep all other factors fixed and then update its mean and variance using the update rule specified below.

$$\begin{aligned}
&\frac{dL[q]}{q(b^{(u)})} = 0, \quad q(b_m^{(u)}) = \mathcal{N}(b_m^{(u)}; \mu_m^{(bu)}, {\sigma_m^{(bu)}}^2) \\
&{\sigma_m^{(bu)}}^2 = (\gamma + \lambda|\Omega(m)|)^{-1} \\
&\mu_m^{(bu)} = {\sigma_m^{(bu)}}^2 \Big(\lambda \sum_{n \in \Omega(m)} (r_{mn} - {\mu_m^{(u)}}^T \mu_n^{(v)} - \mu_n^{(bv)})\Big)
\end{aligned}$$

$$\begin{aligned}
&\frac{dL[q]}{dq(u_m)} = 0, \quad q(u_m) = \mathcal{N}(u_m; \mu_m^{(u)}, \Sigma_m^{(u)}) \\
&\Sigma_m^{(u)} = \Big(\lambda \sum_{n \in \Omega(m)} (\Sigma_n^{(v)} + \mu_n^{(v)} {\mu_n^{(v)}}^T) + \tau I\Big)^{-1} \\
&\mu_m^{(u)} = \Sigma_m^{(u)} \Big(\lambda \sum_{n \in \Omega(m)} (r_{mn} - \mu_m^{(bu)} - \mu_n^{(bv)}) \mu_n^{(v)}\Big)
\end{aligned}$$

See the full derivation in the appendix. We observe that the embedding factor described above yields a non-diagonal covariance matrix. To obtain a fully factorised form, we approximate this factor as follows:

$$\begin{aligned}
q(u_m) &\approx \prod_{k=1}^{K} \mathcal{N}(u_{mk}; \mu_{mk}^{(u)}, {\sigma_{mk}^{(u)}}^2) \\
({\sigma_{mk}^{(u)}}^2)_{k=1}^{K} &= diag({\Sigma_m^{(u)}}^{-1})^{-1}
\end{aligned}$$

Notice that the above approximation only affects the covariance matrix, allowing for a more memory-efficient implementation, particularly for high embedding dimensions. See the appendix for more details. The training algorithm is shown in Algorithm 2.

### 4.3. Feature integration

In (Koenigstein & Paquet, 2013), the authors proposed a way to integrate item features into the model by defining better priors for the items. We used a similar approach by assuming the following:

$$\begin{aligned}
r_{mn}|u_m, v_n, b_m^{(u)}, b_n^{(v)} &\sim \mathcal{N}(r_{mn}; u_m^T v_n + b_m^{(u)} + b_n^{(v)}, \lambda^{-1}) \\
u_m &\sim \mathcal{N}(u_m; 0, \tau^{-1}) \\
v_n &\sim \mathcal{N}(v_n; \frac{1}{\sqrt{|\Gamma(n)|}} \sum_{l \in \Gamma(n)} f_l, \tau^{-1}) \\
f_l &\sim \mathcal{N}(f_l; 0, \tau^{-1}) \\
b_m^{(u)} &\sim \mathcal{N}(b_m^{(u)}; 0, \gamma^{-1}) \\
b_n^{(v)} &\sim \mathcal{N}(b_n^{(v)}; 0, \gamma^{-1})
\end{aligned}$$

**Algorithm 2** Mean Field Variation Update

**Input:** $D = \{(\text{user}, \text{movie}, \text{rating})\}$, $\mathbf{MU}[M, K]$ (User embedding mean matrix), $\mathbf{Mb}_u[M]$ (User bias mean vector), $\mathbf{MV}[N, K]$ (Movie embedding mean matrix), $\mathbf{Mb}_m[N]$ (Movie bias mean vector), $\mathbf{SU}[M, K]$ (User embedding variance matrix), $\mathbf{Sb}_u[M]$ (User bias variance vector), $\mathbf{SV}[N, K]$ (Movie embedding variance matrix), $\mathbf{Sb}_m[N]$ (Movie bias variance vector), $I$ (number of iteration)
**for** iter $i = 0$ **to** $I$ **do**
  **for** user $m = 1$ **to** $M$ **do**
    Update $\mathbf{Mb}_u[m]$ using (movie, rating) from the user $m$ ratings **in** $D$
    Update $\mathbf{Sb}_u[m]$ using (movie, rating) from the user $m$ ratings **in** $D$
  **end for**
  **for** user $m = 1$ **to** $M$ **do**
    Update $\mathbf{MU}[m, :]$ using (movie, rating) from the user $m$ ratings **in** $D$
    Update $\mathbf{SU}[m, :]$ using (movie, rating) from the user $m$ ratings **in** $D$
  **end for**
  **for** movie $n = 1$ **to** $N$ **do**
    Update $\mathbf{Mb}_v[n]$ using (user, rating) from the movie $n$ ratings **in** $D$
    Update $\mathbf{Sb}_v[n]$ using (user, rating) from the movie $n$ ratings **in** $D$
  **end for**
  **for** movie $n = 1$ **to** $N$ **do**
    Update $\mathbf{MU}[n, :]$ using (user, rating) from the movie $n$ ratings **in** $D$
    Update $\mathbf{SU}[n, :]$ using (user, rating) from the movie $n$ ratings **in** $D$
  **end for**
**end for**

We modified the movie prior by incorporating features which, in our case, are movie genres. For a genre $l$, $f_l \in \mathbb{R}$ is defined as its embedding vector. We also define $F = \{f_l\}_{l=1}^{L}$ as the set of all genres and $\Gamma(n)$ the set of all genres to which a movie $n$ belongs.

We are left with a new cost function to minimise:

$$
\begin{aligned}
Cost &= -log(P(U, V, F, b^{(u)}, b^{(v)}|R) \\
Cost &= \frac{\lambda}{2}\sum_{m=1}^{M}\sum_{n\in\Omega(m)}(r_{mn} + u_m^T v_n - b_m^{(u)} + b_n^{(v)})^2 \\
&\quad + \frac{\tau}{2}\sum_{m=1}^{M} u_m^T u_m + \frac{\tau}{2}\sum_{l=1}^{L} f_l^T f_l \\
&\quad + \frac{\tau}{2}\sum_{n=1}^{N}\Big(v_n - \frac{1}{\sqrt{|\Gamma(n)|}}\sum_{l\in\Gamma(n)} f_l\Big)^T \\
&\quad \times \Big(v_n - \frac{1}{\sqrt{|\Gamma(n)|}}\sum_{l\in\Gamma(n)} f_l\Big) \\
&\quad + \frac{\gamma}{2}\sum_{m=1}^{M}(b_m^{(u)})^2 + \frac{\gamma}{2}\sum_{n=1}^{N}(b_n^{(v)})^2 + C
\end{aligned}
$$

We then minimise the cost using alternating least square but with the addition of also updating the feature embedding. While the update rules for the biases and the user embedding do not change from the previous derivation, the expression of the update is now different, and we also have to derive the feature update.

$$
\begin{aligned}
&\frac{dCost}{dv_n} = 0, \\
&\quad v_n = M\Big(\lambda \sum_{n\in\Omega(m)} (r_{mn} - b_m^{(u)} - b_n^{(v)})u_m + \rho\Big)
\end{aligned}
$$

with,

$$
\begin{aligned}
M &= (\lambda \sum_{n\in\Omega(m)} u_m u_m^T + \tau I)^{-1} \\
\rho &= \frac{\tau}{\sqrt{|\Gamma(n)|}}\sum_{l\in\Gamma(n)} f_l
\end{aligned}
$$

$$
\begin{aligned}
&\frac{dCost}{df_l} = 0, \\
&\quad f_l = \alpha\Big(\sum_{n\in\Gamma^{-1}(l)} \frac{1}{\sqrt{|\Gamma(n)|}}\Big(v_n - \phi\Big)\Big)
\end{aligned}
$$

with,

$$
\begin{aligned}
\alpha &= \Big(\sum_{n\in\Gamma^{-1}(l)} \frac{1}{|\Gamma(n)|} + 1\Big)^{-1} \\
\phi &= \frac{1}{\sqrt{|\Gamma(n)|}}\sum_{l'\in\Gamma(n), l'\neq l} f_{l'}
\end{aligned}
$$

See the appendix for the full derivation. The training algorithm is shown in Algorithm 3

## 5. Results

### 5.1. Implementation details

Beyond the naive for-loop implementation of the updates, in order to reduce training time, several optimisations have been introduced. First, the inner training loop consisting of iterating all other users and movies has been parallelised

**Algorithm 3** Alternating Least Square with Feature update

**Input:** $D = \{(\text{user}, \text{movie}, \text{rating})\}$, $G = \{(\text{movie}, \text{genres})\}$, $\mathbf{U}[M, K]$ (User embedding matrix), $\mathbf{b}_u[M]$ (User bias vector), $\mathbf{V}[N, K]$ (Movie embedding matrix), $\mathbf{b}_m[N]$ (Movie bias vector), $\mathbf{F}[L, K]$ (Feature embedding matrix), $I$ (number of iteration)
**for** iter $i = 0$ **to** $I$ **do**
  **for** user $m = 1$ **to** $M$ **do**
    Update $\mathbf{b}_u[m]$ using (movie, rating) from the user $m$ ratings **in** $D$
    Update $\mathbf{U}[m, :]$ using (movie, rating) from the user $m$ ratings **in** $D$
  **end for**
  **for** movie $n = 1$ **to** $N$ **do**
    Update $\mathbf{b}_v[n]$ using (user, rating) from the movie $n$ ratings **in** $D$ and (genre) from movie $n$ genres **in** $G$
    Update $\mathbf{U}[n, :]$ using (user, rating) from the movie $n$ ratings **in** $D$ and (genre) from movie $n$ genres **in** $G$
  **end for**
  **for** movie $l = 1$ **to** $L$ **do**
    Update $\mathbf{F}[n, :]$ using (movie, genres) **in** $G$ where $l$ in genres
  **end for**
**end for**

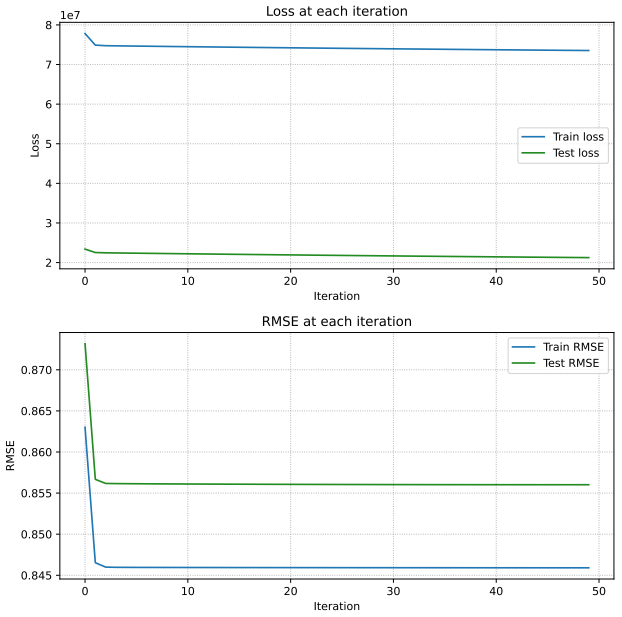


*Figure 4.* Training Bias only

through multiple CPU cores, allowing for updating multiple embeddings and biases in parallel. This is possible due to the fact that the update of a user embedding does not depend on the other user embeddings, and the same goes for user bias, movie bias and movie embedding. For the case of updating features, in order to make use of parallelisation, we computed the updates of all features first and only then applied them. This is to prevent potential inconsistency during training, as the update of a feature is dependent on other features.

Furthermore, to optimise each update, we used vectorisation to optimise single-core computation by converting the majority of the sums into matrix and vector operations, which are accelerated by SIMD computations.

### 5.2. Model training

The data was randomly split into training and test splits, with 80% of the data made into training data and 20% into testing data.

To find the ideal hyperparameters for training the models, we used Bayesian optimisation (Snoek et al., 2012; Nogueira, 2014–) while fixing $K = 2$. The hyperparameters found will then be used to train all the subsequent models. One could argue that selecting the optimal hyperparameters based on $K = 2$ may not be optimal for other embedding sizes; however, this approach was chosen for the sake of time efficiency and to reduce computational cost. All models are trained using 50 iterations, as throughout the experimentation, it has been observed to be enough to reach sufficient and acceptable results.

Figure 4 shows the loss function curve and RMSE for the train and test datasets for a simple model (no variation and no features) trained with bias only. We can see that the training loss curve effectively goes down monotonically, indicating that the training algorithm is actually minimising the loss.

An interesting observation can be made by looking at Figure 5 which shows the train and test loss and RMSE through different embedding dimensions $K$. For the training RMSE, the values steadily decrease as $K$ increases. In contrast, the training loss decreases initially but then begins to increase slightly after $K = 15$, likely due to the increasing effect of regularisation terms in the loss function. A similar pattern can be seen with the test loss, which decreases slightly at $K = 5$ and then gradually increases. However, the most interesting observations can be made when looking at the test RMSE. We can see it decreases at first but then starts to increase after some $K$. This is a sign of overfitting, as the model performs well on the training data but performs poorly on the testing data. We can postulate that the best compromise would come from models with $K$ between 10 and 15.

Figures 7, 8, and 9 show the train and test loss and RMSE for the model trained on $K = 2, 10,$ and 15. Using $K = 2$

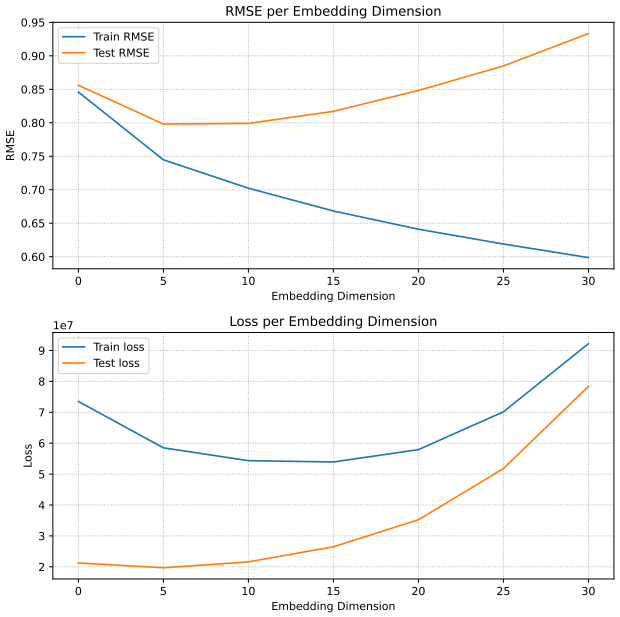


*Figure 5.* Loss and RMSE over embedding dimensions

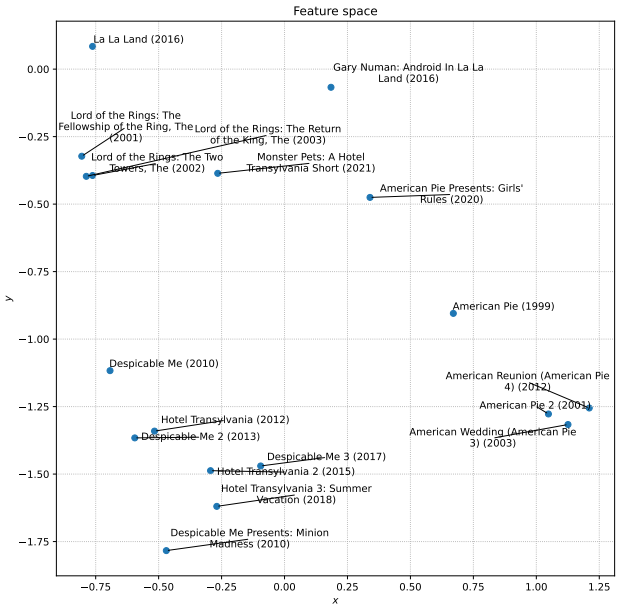


*Figure 6.* Movie Embedding Space for $K = 2$

embeddings, we can visualise the movie feature embedding in 2-dimensional space, as shown in Figure 6. We restricted the visualisation to a few movies to show that sequels and prequels, as well as related films, are located near each other in the space, whereas contrasting movies appear in opposite regions of the space. We can see that the adventure and fantasy *Lord of the Rings* series are distant from the comedy *American Pie* series, while the animated films *Despicable Me* and *Hotel Transylvania* are close together.

Figures 10, 11, 12, show the train and test loss and RMSE of the model with features included trained on $K = 2, 10,$ and 15. Figure 13 shows us the 2-dimensional feature embedding space. Due to the imbalanced nature of the genre distribution as seen in Figure 3, some embeddings might be hard to reason with. We also have to take into account that errors and inconsistencies might be present in the genre labelling of the titles, as mentioned on the dataset website (GroupLens). However, we can explain some cases, such as the fact that 'war' and 'western' are close in space probably because they are similar, whereas 'children' and 'horror' are separated in space, most likely because most horror films are not appropriate for children to watch.

For the model using variational inference, we can see the train and test $L[q]$ (ELBO) and RMSE in Figures 14, 15 and 16 of the models trained on $K = 2, 10,$ and 15. Note that the RMSE values are calculated using the means of the user and movie embeddings and biases distributions to predict ratings. This is equivalent to evaluating the rating as the expectation of a user's rating distribution for a movie. This

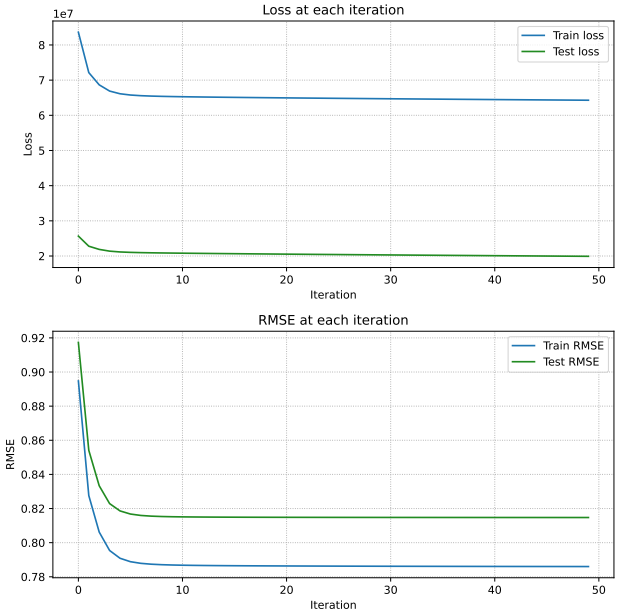


*Figure 7.* Training with $K = 2$ embedding

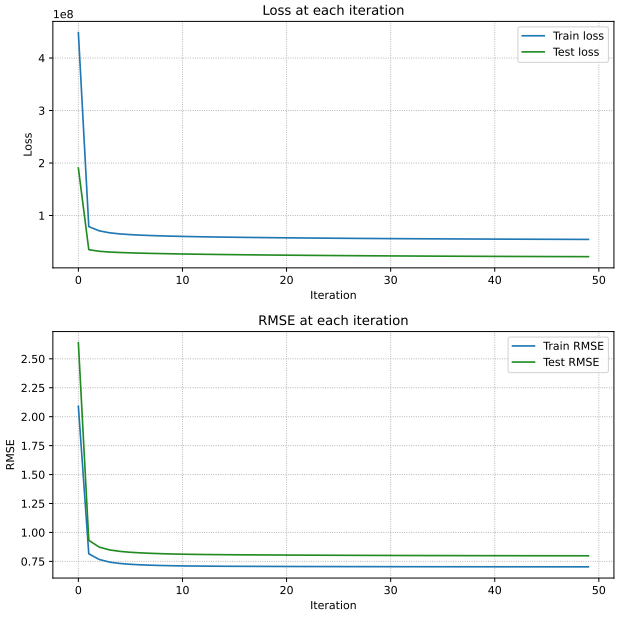


*Figure 8.* Training with $K = 10$ embedding

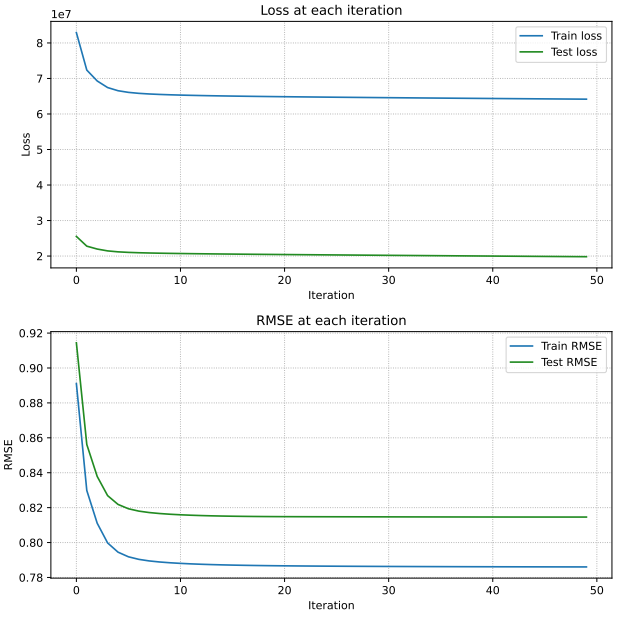


*Figure 10.* Training with $K = 2$ embedding with features

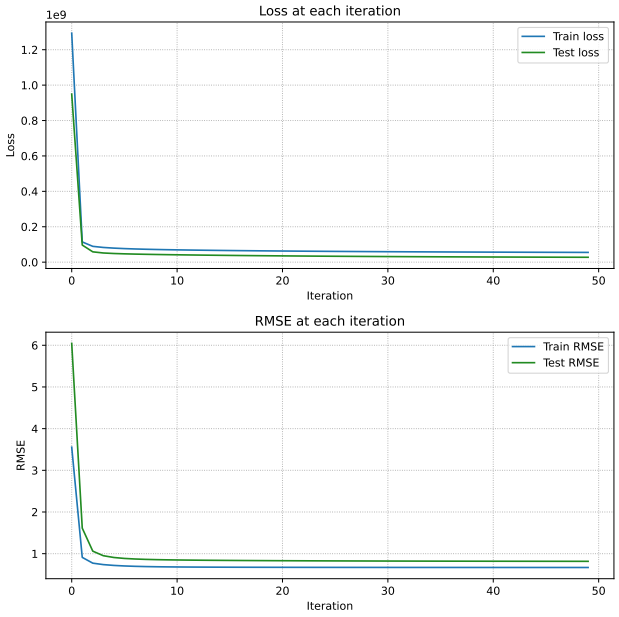


*Figure 9.* Training with $K = 15$ embedding

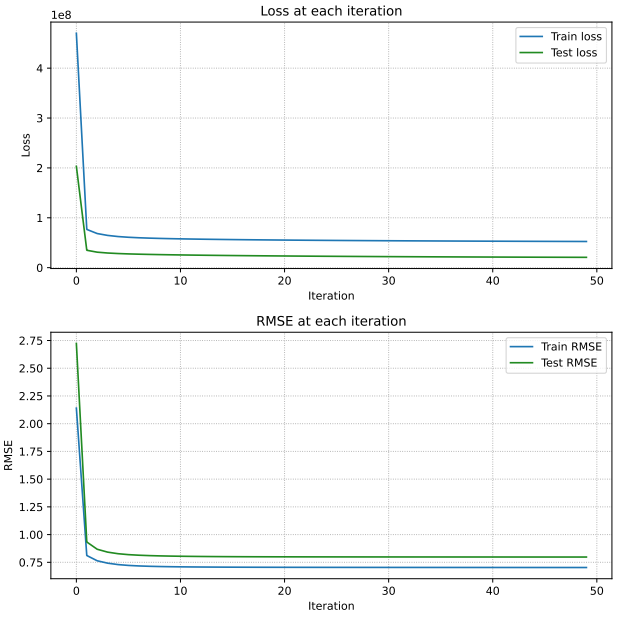


*Figure 11.* Training with $K = 10$ embedding with features

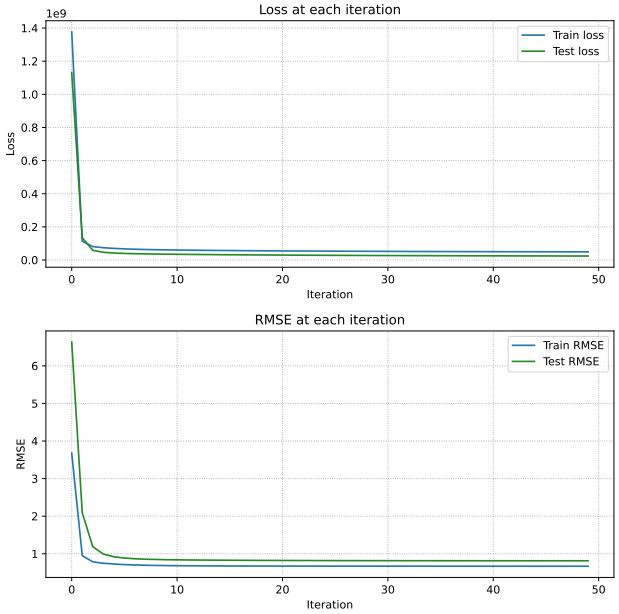


*Figure 12.* Training with $K = 15$ embedding with features

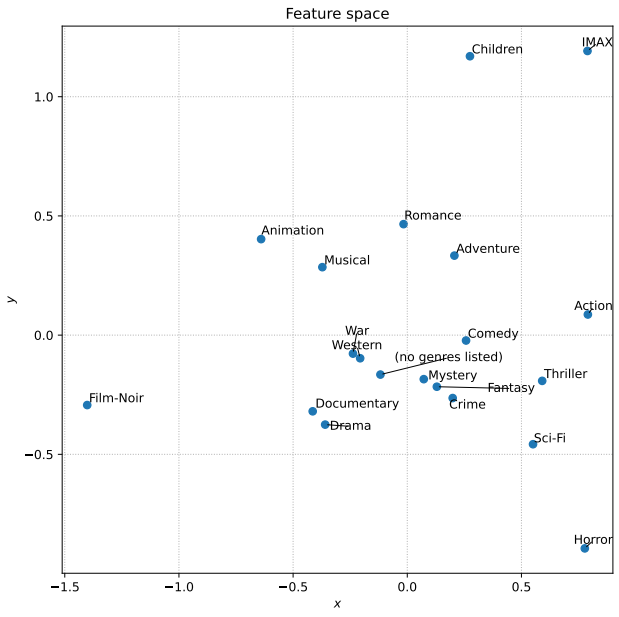


*Figure 13.* Feature Embedding Space for $K = 2$

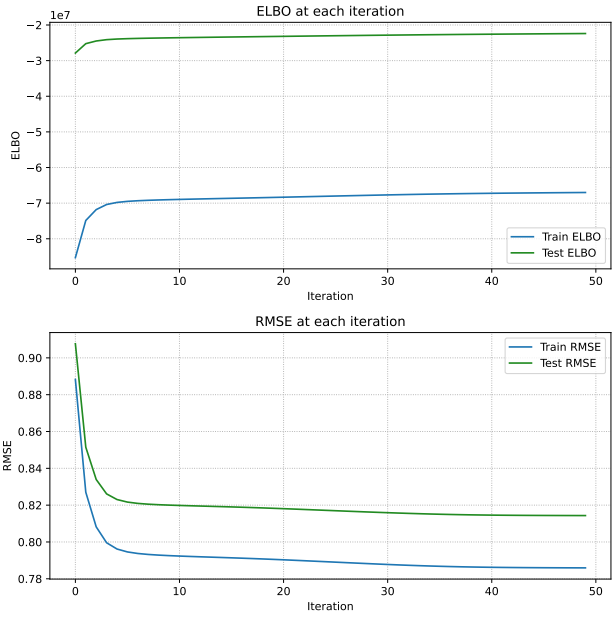


*Figure 14.* Variational Inference $K = 2$

*Table 1.* Model Training Results

| MODEL | TRAIN RMSE | TEST RMSE |
|---|---|---|
| BIAS ONLY | 0.85 | 0.86 |
| K=2 | 0.79 | 0.81 |
| K=10 | 0.70 | 0.80 |
| K=15 | 0.67 | 0.82 |
| K=2 WITH GENRES | 0.79 | 0.82 |
| K=10 WITH GENRES | 0.70 | 0.80 |
| K=15 WITH GENRES | 0.67 | 0.81 |
| K=2 VARIATIONAL | 0.79 | 0.81 |
| K=10 VARIATIONAL | 0.70 | 0.79 |
| K=15 VARIATIONAL | 0.67 | 0.81 |

is shown in the appendix.

Table 1 compares the train and test RMSE across all implemented models. We notice that our three approaches perform similarly under the same configuration.

### 5.3. Inference

We tested the $K = 10$ models for the prediction task. We created a dummy user that gave a 5.0 rating to the movie *Lord of the Rings: The Fellowship of the Ring* by Peter Jackson, which is an adventure and fantasy movie. We then computed the embedding and bias of that dummy user using a few alternating least squares iterations. When evaluating the score, we used the following expression:

$$score_{mn} = \text{user\_embedding}[m]^T \text{movie\_embedding}[n] + \text{movie\_bias}[n]$$

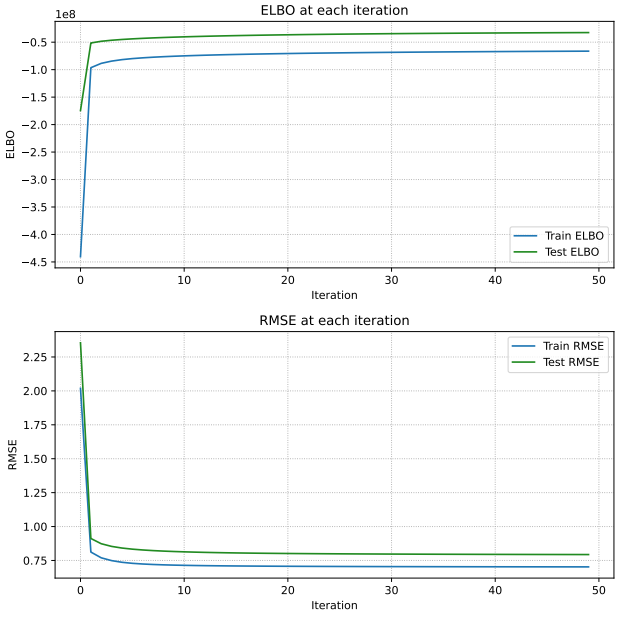


*Figure 15.* Variational Inference $K = 10$

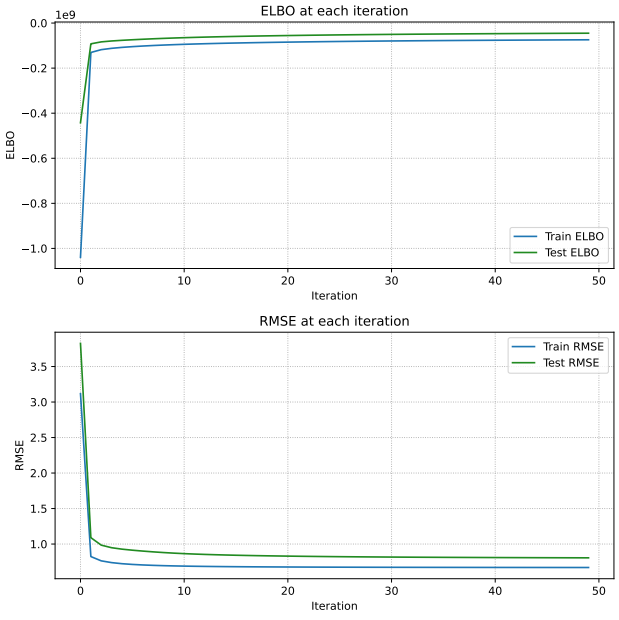


*Figure 16.* Variational Inference $K = 15$

For the case of the variational model, we estimate the score using the following expression:

$$\begin{aligned} score_{mn} = &\text{ user\_embedding\_mean}[m]^T \\ &\cdot \text{movie\_embedding\_mean}[n] \\ &+ \text{movie\_bias\_mean}[n] \end{aligned}$$

At first, we came across poor results where the rated movie was not included in the top 10 for all tested models. We noticed that the top predictions are movies that have only been rated a few times, which shows that those results are due to overfitting. To make the prediction better, we filtered all movies that have fewer $10$ ratings when making the prediction. Also, to diminish the impact of high bias on prediction, a factor $0.05$ has been multiplied to the movie bias in the expression above when evaluating the score. Tables 2, 3, and 4 show the final rankings for base, feature, and variational models, respectively.

## 6. Conclusion

In this paper, we explored several collaborative filtering approaches to recommender systems, with a focus on matrix factorisation techniques. We began by formulating the basic matrix factorisation model within a probabilistic framework. We then investigated more advanced methods, including a variational inference approach designed to better handle datasets exhibiting a power-law distribution of interactions and a feature-integrated model that incorporates movie genres to address cold-start scenarios. Empirical evaluation revealed that while these models learn meaningful latent representations they remain susceptible to overfitting on popular items, necessitating post-processing steps such as filtering low-degree movies and adjusting bias terms to yield coherent recommendations.

Although the variational method could potentially provide a way to mitigate overfitting, its practical benefit in our evaluation was limited, and post-processing was still required. Moreover, our current test setting does not fully capture the qualitative improvements in recommendation diversity or robustness for tail items. While the variational framework offers a full predictive distribution, we only used its expectation as a point estimate for scoring; future work could explore a better use of the whole probability distribution by leveraging uncertainty for ranking or making confidence-aware recommendations. Additionally, we did not quantitatively assess how effectively the feature-integrated model mitigates item cold-start. Furthermore, future work could also explore incorporating user-side metadata to address user cold-start.

*Table 2.* Base Model Top-10 Results

| RANK | TITLE (YEAR) | DEGREE | GENRES |
|---|---|---|---|
| 1 | *Lord of the Rings: The Return of the King* (2003) | 67449 | ACTION, ADVENTURE, DRAMA, FANTASY |
| 2 | *Lord of the Rings: The Fellowship of the Ring* (2001) | 73122 | ADVENTURE, FANTASY |
| 3 | *Lord of the Rings: The Two Towers* (2002) | 67463 | ADVENTURE, FANTASY |
| 4 | *Star Wars: Episode V – The Empire Strikes Back* (1980) | 72151 | ACTION, ADVENTURE, SCI-FI |
| 5 | *Star Wars: Episode IV – A New Hope* (1977) | 85010 | ACTION, ADVENTURE, SCI-FI |
| 6 | *Star Wars: Episode VI – Return of the Jedi* (1983) | 67496 | ACTION, ADVENTURE, SCI-FI |
| 7 | *Harry Potter and the Deathly Hallows: Part 2* (2011) | 20227 | ACTION, ADVENTURE, DRAMA, FANTASY, MYSTERY, IMAX |
| 8 | *Star Wars: Episode III – Revenge of the Sith* (2005) | 24773 | ACTION, ADVENTURE, SCI-FI |
| 9 | *Harry Potter and the Deathly Hallows: Part 1* (2010) | 20969 | ACTION, ADVENTURE, FANTASY, IMAX |
| 10 | *Harry Potter and the Half-Blood Prince* (2009) | 21033 | ADVENTURE, FANTASY, MYSTERY, ROMANCE, IMAX |

*Table 3.* Model with features Top-10 Results

| RANK | TITLE (YEAR) | DEGREE | GENRES |
|---|---|---|---|
| 1 | *Lord of the Rings: The Return of the King* (2003) | 67449 | ACTION, ADVENTURE, DRAMA, FANTASY |
| 2 | *Lord of the Rings: The Two Towers* (2002) | 67463 | ADVENTURE, FANTASY |
| 3 | *Lord of the Rings: The Fellowship of the Ring* (2001) | 73122 | ADVENTURE, FANTASY |
| 4 | *Star Wars: Episode III – Revenge of the Sith* (2005) | 24773 | ACTION, ADVENTURE, SCI-FI |
| 5 | *Hobbit: An Unexpected Journey* (2012) | 16149 | ADVENTURE, FANTASY, IMAX |
| 6 | *Hobbit: The Desolation of Smaug* (2013) | 12175 | ADVENTURE, FANTASY, IMAX |
| 7 | *Star Wars: Episode IV – A New Hope* (1977) | 85010 | ACTION, ADVENTURE, SCI-FI |
| 8 | *Star Wars: Episode VI – Return of the Jedi* (1983) | 67496 | ACTION, ADVENTURE, SCI-FI |
| 9 | *Star Wars: Episode V - The Empire Strikes Back* (1980) | 72151 | ACTION, ADVENTURE, SCI-FI |
| 10 | *Star Wars: Episode II – Attack of the Clones* (2002) | 27415 | ACTION, ADVENTURE, SCI-FI, IMAX |

## References


Barabási, A.-L. and Pósfai, M. *Network Science*. Cambridge University Press, 2016. ISBN 9781107076266. URL https://networksciencebook.com/.

Bishop, C. M. Approximate inference. In *Pattern Recognition and Machine Learning*, chapter 10. Springer, 2006. URL https://www.microsoft.com/en-us/research/publication/pattern-recognition-machine-learning/.

GroupLens. Movielens 32m dataset readme. https://files.grouplens.org/datasets/movielens/ml-32m-README.html. Accessed: 2025-11-23.

Harper, F. M. and Konstan, J. A. The movielens datasets: History and context. *ACM Transactions on Interactive Intelligent Systems (TiiS)*, 5(4):19:1–19:19, 2015. doi: 10.1145/2827872. URL https://doi.org/10.1145/2827872.

Koenigstein, N. and Paquet, U. Xbox movies recommendations: Variational bayes matrix factorization with embedded feature selection. pp. 129–136, 10 2013. doi: 10.1145/2507157.2507168.

Koenigstein, N., Nice, N., Paquet, U., and Schleyen, N. The xbox recommender system. 09 2012. doi: 10.1145/2365952.2366015.

Koren, Y., Bell, R., and Volinsky, C. Matrix factorization techniques for recommender systems. *Computer*, 42(8): 30–37, 2009. doi: 10.1109/MC.2009.263.

Nogueira, F. Bayesian Optimization: Open source constrained global optimization tool for Python, 2014–. URL https://github.com/bayesian-optimization/BayesianOptimization.

Snoek, J., Larochelle, H., and Adams, R. P. Practical bayesian optimization of machine learning algorithms, 2012. URL https://arxiv.org/abs/1206.2944.

*Table 4.* Variational Model Top-10 Results

| RANK | TITLE (YEAR) | DEGREE | GENRES |
|---|---|---|---|
| 1 | *Lord of the Rings: The Return of the King* (2003) | 67449 | ACTION, ADVENTURE, DRAMA, FANTASY |
| 2 | *Lord of the Rings: The Two Towers* (2002) | 67463 | ADVENTURE, FANTASY |
| 3 | *Lord of the Rings: The Fellowship of the Ring* (2001) | 73122 | ADVENTURE, FANTASY |
| 4 | *Star Wars: Episode III – Revenge of the Sith* (2005) | 24773 | ACTION, ADVENTURE, SCI-FI |
| 5 | *Star Wars: Episode V - The Empire Strikes Back* (1980) | 72151 | ACTION, ADVENTURE, SCI-FI |
| 6 | *Star Wars: Episode IV – A New Hope* (1977) | 85010 | ACTION, ADVENTURE, SCI-FI |
| 7 | *Star Wars: Episode VI – Return of the Jedi* (1983) | 67496 | ACTION, ADVENTURE, SCI-FI |
| 8 | *Star Wars: Episode II – Attack of the Clones* (2002) | 27415 | ACTION, ADVENTURE, SCI-FI, IMAX |
| 9 | *Hobbit: An Unexpected Journey* (2012) | 16149 | ADVENTURE, FANTASY, IMAX |
| 10 | *Hobbit: The Desolation of Smaug* (2013) | 12175 | ADVENTURE, FANTASY, IMAX |

# A. Appendix

## A.1. Expectation and Entropy

Let $U$, $V$ independent random variables and $A$ a matrix. We take note of the following expressions:

$$\begin{aligned}
E[U^T AU] &= E[tr(U^T AU)] \\
&= E[tr(AUU^T)] \\
&= tr(AE[UU^T]) \\
&= tr(A(Var(U) + E[U]E[U]^T)) \\
&= tr(A(Var(U)) + tr(AE[U]E[U]^T)) \\
&= tr(A(Var(U)) + tr(E[U]^T AE[U])) \\
E[U^T AU] &= tr(A(Var(U))) + E[U]^T AE[U]
\end{aligned}$$

$$\begin{aligned}
E[(U^T V)^2] &= E[(U^T V)^T (U^T V)] \\
&= E[V^T UU^T V] \\
&= E[tr(V^T UU^T V)] \\
&= E[tr(UU^T VV^T)] \\
&= tr(E[UU^T]E[VV^T]) \\
E[(U^T V)^2] &= tr((Var(U) + E[U]E[U]^T)(Var(V) + E[V]E[V]^T))
\end{aligned}$$

Let $q(U) \sim N(U; \mu, \Sigma)$. The entropy $H_q = E_q[log(q(U))]$ can be written as follows:

$$\begin{aligned}
E_q[log(q(U))] &= log\Big(\frac{1}{\sqrt{2\pi}^K |\Sigma|^{\frac{1}{2}}}\Big) - \frac{1}{2} E_q[(u-\mu)^T \Sigma^{-1}(u-\mu)] \\
&= -\frac{K}{2} log(2\pi) - \frac{1}{2} log(|\Sigma|) - \frac{1}{2} E_q\Big[tr\Big((u-\mu)^T \Sigma^{-1}(u-\mu)\Big)\Big] \\
&= -\frac{K}{2} log(2\pi) - \frac{1}{2} log(|\Sigma|) - \frac{1}{2} E_q\Big[tr\Big(\Sigma^{-1}(u-\mu)(u-\mu)^T\Big)\Big] \\
&= -\frac{K}{2} log(2\pi) - \frac{1}{2} log(|\Sigma|) - \frac{1}{2} tr\Big(\Sigma^{-1} E_q[(u-\mu)(u-\mu)^T]\Big) \\
&= -\frac{K}{2} log(2\pi) - \frac{1}{2} log(|\Sigma|) - \frac{1}{2} tr\Big(\Sigma^{-1}\Sigma\Big)
\end{aligned}$$

$$E_q[log(q(U))] = -\frac{K}{2}log(2\pi) - \frac{1}{2}log(|\Sigma|) - \frac{K}{2}$$

### A.2. Cost and Updates rule

The cost is defined as follows:

$$\begin{aligned}
Cost &= -log(P(U, V, b^{(u)}, b^{(v)}|R) \\
&= -log(P(R|U, V, b^{(u)}, b^{(v)}) - log(P(U)) - log(P(V)) - log(P(b^{(u)})) - log(P(b^{(v)})) \\
Cost &= \frac{\lambda}{2}\sum_{m=1}^{M}\sum_{n\in\Omega(m)}(r_{mn} + u_m^T v_n - b_m^{(u)} + b_n^{(v)})^2 \\
&\quad + \frac{\tau}{2}\sum_{m=1}^{M} u_m^T u_m + \frac{\tau}{2}\sum_{n=1}^{N} v_n^T v_n \\
&\quad + \frac{\gamma}{2}\sum_{m=1}^{M}(b_m^{(u)})^2 + \frac{\gamma}{2}\sum_{n=1}^{N}(b_n^{(v)})^2 + C
\end{aligned}$$

From this we can derive the user embedding update (similar expression for movie embedding update)

$$\begin{aligned}
\frac{dCost}{du_m} &= -\lambda\sum_{n\in\Omega(m)}(r_{mn} - u_m^T v_n - b_m^{(u)} - b_n^{(v)})v_n + \tau u_m \\
&= -\lambda\sum_{n\in\Omega(m)}(r_{mn} - b_m^{(u)} - b_n^{(v)})v_n + \lambda\sum_{n\in\Omega(m)}(u_m^T v_n)v_n + \tau u_m \\
&= -\lambda\sum_{n\in\Omega(m)}(r_{mn} - b_m^{(u)} - b_n^{(v)})v_n + \lambda\sum_{n\in\Omega(m)}(v_n v_n^T)u_m + \tau u_m \\
\frac{dCost}{du_m} &= -\lambda\sum_{n\in\Omega(m)}(r_{mn} - b_m^{(u)} - b_n^{(v)})v_n + (\lambda\sum_{n\in\Omega(m)} v_n v_n^T + \tau I)u_m \\
\frac{dCost}{du_m} = 0, \quad u_m &= (\lambda\sum_{n\in\Omega(m)} v_n v_n^T + \tau I)^{-1}(\lambda\sum_{n\in\Omega(m)}(r_{mn} - b_m^{(u)} - b_n^{(v)})v_n)
\end{aligned}$$

We can then derive the user bias update (similar expression for movie bias update)

$$\begin{aligned}
\frac{dCost}{db_m^{(u)}} &= -\lambda\sum_{n\in\Omega(m)}(r_{mn} - u_m^T v_n - b_m^{(u)} - b_n^{(v)}) + \gamma(b_m^{(u)}) \\
&= -\lambda\sum_{n\in\Omega(m)}(r_{mn} - u_m^T v_n - b_n^{(v)}) + \lambda\sum_{n\in\Omega(m)} b_m^{(u)} + \gamma(b_m^{(u)}) \\
\frac{dCost}{db_m^{(u)}} &= -\lambda\sum_{n\in\Omega(m)}(r_{mn} - u_m^T v_n - b_n^{(v)}) + (\lambda|\Omega(m)| + \gamma)b_m^{(u)} \\
\frac{dCost}{db_m^{(u)}} = 0, \quad b_m^{(u)} &= \frac{\lambda\sum_{n\in\Omega(m)}(r_{mn} - u_m^T v_n - b_n^{(v)})}{\lambda|\Omega(m)| + \gamma}
\end{aligned}$$

### A.3. Cost and Update rules with feature

When integrating features, the cost becomes,

$$
\begin{aligned}
Cost &= -log(P(U,V,F,b^{(u)},b^{(v)}|R) \\
&= -log(P(R|U,V,F,b^{(u)},b^{(v)}) - log(P(U)) - log(P(V)) - log(P(F)) - log(P(b^{(u)})) - log(P(b^{(v)})) \\
Cost &= \frac{\lambda}{2}\sum_{m=1}^{M}\sum_{n\in\Omega(m)}(r_{mn}+u_m^T v_n - b_m^{(u)} + b_n^{(v)})^2 \\
&\quad + \frac{\tau}{2}\sum_{m=1}^{M} u_m^T u_m + \frac{\tau}{2}\sum_{l=1}^{L} f_l^T f_l \\
&\quad + \frac{\tau}{2}\sum_{n=1}^{N}\Big(v_n - \frac{1}{\sqrt{|\Gamma(n)|}}\sum_{l\in\Gamma(n)} f_l\Big)^T\Big(v_n - \frac{1}{\sqrt{|\Gamma(n)|}}\sum_{l\in\Gamma(n)} f_l\Big) \\
&\quad + \frac{\gamma}{2}\sum_{m=1}^{M}(b_m^{(u)})^2 + \frac{\gamma}{2}\sum_{n=1}^{N}(b_n^{(v)})^2 + C
\end{aligned}
$$

The user embedding, user bias and movie bias updates stay the same, but the movie embedding update becomes:

$$
\begin{aligned}
\frac{dCost}{dv_n} &= -\lambda\sum_{n\in\Omega(m)}(r_{mn} - u_m^T v_n - b_m^{(u)} - b_n^{(v)})u_m + \tau\Big(v_n - \frac{1}{\sqrt{|\Gamma(n)|}}\sum_{l\in\Gamma(n)} f_l\Big) \\
&= -\lambda\sum_{n\in\Omega(m)}(r_{mn} - b_m^{(u)} - b_n^{(v)})u_m + \lambda\sum_{n\in\Omega(m)}(u_m^T v_n)u_m + \tau\Big(v_n - \frac{1}{\sqrt{|\Gamma(n)|}}\sum_{l\in\Gamma(n)} f_l\Big) \\
&= -\lambda\sum_{n\in\Omega(m)}(r_{mn} - b_m^{(u)} - b_n^{(v)})u_m + \lambda\sum_{n\in\Omega(m)}(u_m u_m^T)v_n + \tau\Big(v_n - \frac{1}{\sqrt{|\Gamma(n)|}}\sum_{l\in\Gamma(n)} f_l\Big) \\
\frac{dCost}{dv_n} &= -\Big(\lambda\sum_{n\in\Omega(m)}(r_{mn} - b_m^{(u)} - b_n^{(v)})u_m + \frac{\tau}{\sqrt{|\Gamma(n)|}}\sum_{l\in\Gamma(n)} f_l\Big) + (\lambda\sum_{n\in\Omega(m)} u_m u_m^T + \tau I)v_n \\
\frac{dCost}{dv_n} = 0, \quad v_n &= (\lambda\sum_{n\in\Omega(m)} u_m u_m^T + \tau I)^{-1}\Big(\lambda\sum_{n\in\Omega(m)}(r_{mn} - b_m^{(u)} - b_n^{(v)})u_m + \frac{\tau}{\sqrt{|\Gamma(n)|}}\sum_{l\in\Gamma(n)} f_l\Big)
\end{aligned}
$$

We then derive the feature embedding update as follows:

$$
\begin{aligned}
\frac{dCost}{df_l} &= -\tau\sum_{n\in\Gamma^{-1}(l)}\frac{1}{\sqrt{|\Gamma(n)|}}\Big(v_n - \frac{1}{\sqrt{|\Gamma(n)|}}\sum_{l'\in\Gamma(n)} f_{l'}\Big) + \tau f_l \\
&= -\tau\sum_{n\in\Gamma^{-1}(l)}\frac{1}{\sqrt{|\Gamma(n)|}}v_n + \tau\sum_{n\in\Gamma^{-1}(l)}\frac{1}{|\Gamma(n)|}\sum_{l'\in\Gamma(n)} f_{l'} + \tau f_l \\
\frac{dCost}{df_l} &= -\tau\Big(\sum_{n\in\Gamma^{-1}(l)}\frac{1}{\sqrt{|\Gamma(n)|}}v_n - \sum_{n\in\Gamma^{-1}(l)}\frac{1}{|\Gamma(n)|}\sum_{l'\in\Gamma(n),l'\neq l} f_{l'}\Big) + \tau\Big(\sum_{n\in\Gamma^{-1}(l)}\frac{1}{|\Gamma(n)|} + 1\Big)f_l \\
\frac{dCost}{df_l} = 0, \quad f_l &= \Big(\sum_{n\in\Gamma^{-1}(l)}\frac{1}{|\Gamma(n)|} + 1\Big)^{-1}\Big(\sum_{n\in\Gamma^{-1}(l)}\frac{1}{\sqrt{|\Gamma(n)|}}\Big(v_n - \frac{1}{\sqrt{|\Gamma(n)|}}\sum_{l'\in\Gamma(n),l'\neq l} f_{l'}\Big)\Big)
\end{aligned}
$$

### A.4. Mean Field Approximation

#### A.4.1. Minimising the KL Divergence

The KL Divergence can be expressed as:

$$
\begin{aligned}
D_{KL}(q(U)q(V)q(b^{(u)})q(b^{(v)})||P(U,V,b^{(u)},b^{(v)}|R)) &= E_{q(U)q(V)q(b^{(u)})q(b^{(v)})}\left[log\left(\frac{q(U)q(V)q(b^{(u)})q(b^{(v)})}{P(U,V,b^{(u)},b^{(v)}|R)}\right)\right] \\
&= E_{q(U)q(V)q(b^{(u)})q(b^{(v)})}\left[log\left(\frac{q(U)q(V)q(b^{(u)})q(b^{(v)})P(R)}{P(R,U,V,b^{(u)},b^{(v)})}\right)\right] \\
&= E_{q(U)q(V)q(b^{(u)})q(b^{(v)})}\left[log\left(\frac{q(U)q(V)q(b^{(u)})q(b^{(v)})}{P(R,U,V,b^{(u)},b^{(v)})}\right)\right] \\
&\quad + E_{q(U)q(V)q(b^{(u)})q(b^{(v)})}\left[log(P(R))\right] \\
D_{KL}(q(U)q(V)q(b^{(u)})q(b^{(v)})||P(U,V,b^{(u)},b^{(v)}|R)) &= E_{q(U)q(V)q(b^{(u)})q(b^{(v)})}\left[log\left(\frac{q(U)q(V)q(b^{(u)})q(b^{(v)})}{P(R,U,V,b^{(u)},b^{(v)})}\right)\right] + log(P(R))
\end{aligned}
$$

We can then derive the following expressions:

$$
\begin{aligned}
log(P(R)) &= D_{KL}(q(U)q(V)q(b^{(u)})q(b^{(v)})||P(U,V,b^{(u)},b^{(v)}|R)) - E_{q(U)q(V)q(b^{(u)})q(b^{(v)})}\left[log\left(\frac{q(U)q(V)q(b^{(u)})q(b^{(v)})}{P(R,U,V,b^{(u)},b^{(v)})}\right)\right] \\
&= D_{KL}(q(U)q(V)q(b^{(u)})q(b^{(v)})||P(U,V,b^{(u)},b^{(v)}|R)) + E_{q(U)q(V)q(b^{(u)})q(b^{(v)})}\left[log\left(\frac{P(R,U,V,b^{(u)},b^{(v)})}{q(U)q(V)q(b^{(u)})q(b^{(v)})}\right)\right] \\
log(P(R)) &= D_{KL}(q(U)q(V)q(b^{(u)})q(b^{(v)})||P(U,V,b^{(u)},b^{(v)}|R)) + L[q] \\
L[q] &= E_{q(U)q(V)q(b^{(u)})q(b^{(v)})}\left[log\left(\frac{P(R,U,V,b^{(u)},b^{(v)})}{q(U)q(V)q(b^{(u)})q(b^{(v)})}\right)\right] \\
&= E_{q(U)q(V)q(b^{(u)})q(b^{(v)})}\left[log\left(P(R,U,V,b^{(u)},b^{(v)})\right)\right] - E_{q(U)q(V)q(b^{(u)})q(b^{(v)})}\left[log\left(q(U)q(V)q(b^{(u)})q(b^{(v)})\right)\right] \\
L[q] &= \int q(U)\left(E_{q(V)q(b^{(u)})q(b^{(v)})}\left[log\left(P(R,U,V,b^{(u)},b^{(v)})\right)\right]\right)dU \\
&\quad - \int q(U)log\left(q(U)\right) - E_{q(V)q(b^{(u)})q(b^{(v)})}\left[log\left(q(V)q(b^{(u)})q(b^{(v)})\right)\right]
\end{aligned}
$$

Minimising the KL Divergence is equivalent to maximising $L[q]$, hence we have the following update rules.

### A.4.2. User and Movie embeddings update

$$
\begin{aligned}
\frac{dL[q]}{dq(U)} &= E_{q(V)q(b^{(u)})q(b^{(v)})}\left[log\left(P(R,U,V,b^{(u)},b^{(v)})\right)\right] - log\left(q(U)\right) + C \\
\frac{dL[q]}{dq(U)} = 0, \quad log\left(q(U)\right) &= E_{q(V)q(b^{(u)})q(b^{(v)})}\left[log\left(P(R,U,V,b^{(u)},b^{(v)})\right)\right] + C \\
q(U) &= Kexp\left(E_{q(V)q(b^{(u)})q(b^{(v)})}\left[log\left(P(R,U,V,b^{(u)},b^{(v)})\right)\right]\right) \\
&= Kexp\Big(E_{q(V)q(b^{(u)})q(b^{(v)})}\Big[log(P(R|U,V,b^{(u)},b^{(v)}) \\
&\quad + log(P(U)) + log(P(V)) + log(P(b^{(u)})) + log(P(b^{(v)}))\Big]\Big)
\end{aligned}
$$

Absorbing the constant terms into $K$ gives us,

$$
\begin{aligned}
q(U) = K'exp\Big(E_{q(V)q(b^{(u)})q(b^{(v)})}\Big[ &- \frac{\lambda}{2}\sum_{m=1}^{M}\sum_{n\in\Omega(m)}(r_{mn} - u_m^T v_n - b_m^{(u)} - b_n^{(v)})^2 \\
&- \frac{\tau}{2}\sum_{m=1}^{M}u_m^T u_m - \frac{\tau}{2}\sum_{n=1}^{N}v_n^T v_n - \frac{\gamma}{2}\sum_{m=1}^{M}(b_m^{(u)})^2 - \frac{\gamma}{2}\sum_{n=1}^{N}(b_n^{(v)})^2\Big]\Big)
\end{aligned}
$$

$$= K' exp\Big(E_{q(V)q(b^{(u)})q(b^{(v)})}\Big[ - \frac{\lambda}{2}\sum_{m=1}^{M}\sum_{n\in\Omega(m)}(r_{mn}^2 - 2r_{mn}u_m^T v_n - 2r_{mn}b_m^{(u)} - 2r_{mn}b_n^{(v)}$$
$$+ (u_m^T v_n)^2 + 2b_m^{(u)}u_m^T v_n + 2b_n^{(v)}u_m^T v_n + 2b_m^{(u)}b_n^{(v)} + {b_m^{(u)}}^2 + {b_n^{(v)}}^2)$$
$$- \frac{\tau}{2}\sum_{m=1}^{M}u_m^T u_m - \frac{\tau}{2}\sum_{n=1}^{N}v_n^T v_n - \frac{\gamma}{2}\sum_{m=1}^{M}(b_m^{(u)})^2 - \frac{\gamma}{2}\sum_{n=1}^{N}(b_n^{(v)})^2\Big]\Big)$$
$$= K' exp\Big( - \frac{\lambda}{2}\sum_{m=1}^{M}\sum_{n\in\Omega(m)}(r_{mn}^2 - 2r_{mn}u_m^T E_{q(V)}[v_n] - 2r_{mn}E_{b_m^{(u)}}[b_m^{(u)}] - 2r_{mn}E_{b_n^{(v)}}[b_n^{(v)}]$$
$$+ u_m^T E_{q(V)}[v_n v_n^T]u_m + 2E_{b_m^{(u)}}[b_m^{(u)}](u_m^T E_{q(V)}[v_n]) + 2E_{b_n^{(v)}}[b_n^{(v)}](u_m^T E_{q(V)}[v_n])$$
$$+ 2E_{b_m^{(u)}}[b_m^{(u)}]E_{b_n^{(v)}}[b_n^{(v)}] + E_{b_m^{(u)}}[{b_m^{(u)}}^2] + E_{b_n^{(v)}}[{b_n^{(v)}}^2])$$
$$- \frac{\tau}{2}\sum_{m=1}^{M}u_m^T u_m - \frac{\tau}{2}\sum_{n=1}^{N}E_{q(V)}[v_n^T v_n] - \frac{\gamma}{2}\sum_{m=1}^{M}E_{b_m^{(u)}}[{b_m^{(u)}}^2] - \frac{\gamma}{2}\sum_{n=1}^{N}E_{b_n^{(v)}}[{b_n^{(v)}}^2])\Big)$$

Absorbing the terms not depending on $U$ into $K'$ gives us,

$$q(U) = K'' exp\Big( - \frac{\lambda}{2}\sum_{m=1}^{M}\sum_{n\in\Omega(m)}(-2r_{mn}u_m^T E_{q(V)}[v_n] + u_m^T E_{q(V)}[v_n v_n^T]u_m$$
$$+ 2E_{b_m^{(u)}}[b_m^{(u)}](u_m^T E_{q(V)}[v_n]) + 2E_{b_n^{(v)}}[b_n^{(v)}](u_m^T E_{q(V)}[v_n])) - \frac{\tau}{2}\sum_{m=1}^{M}u_m^T u_m\Big)$$
$$= K'' \prod_{m=1}^{M} exp\Big(\sum_{n\in\Omega(m)} - \frac{\lambda}{2}(-2r_{mn}u_m^T E_{q(V)}[v_n] + u_m^T E_{q(V)}[v_n v_n^T]u_m$$
$$+ 2E_{b_m^{(u)}}[b_m^{(u)}](u_m^T E_{q(V)}[v_n]) + 2E_{b_n^{(v)}}[b_n^{(v)}](u_m^T E_{q(V)}[v_n])) - \frac{\tau}{2}u_m^T u_m\Big)$$
$$= K'' \prod_{m=1}^{M} exp\Big(\lambda\sum_{n\in\Omega(m)}(r_{mn}E_{q(V)}[v_n]^T - E_{b_m^{(u)}}[b_m^{(u)}]E_{q(V)}[v_n]^T$$
$$- E_{b_n^{(v)}}[b_n^{(v)}]E_{q(V)}[v_n]^T)u_m - u_m^T\Big(\frac{\lambda}{2}\sum_{n\in\Omega(m)}E_{q(V)}[v_n v_n^T] + \frac{\tau}{2}I\Big)u_m\Big)$$
$$= K'' \prod_{m=1}^{M} exp\Big(\Big(\frac{\lambda}{2}\sum_{n\in\Omega(m)}E_{q(V)}[v_n v_n^T] + \frac{\tau}{2}I\Big)\Big(\Big(\frac{\lambda}{2}\sum_{n\in\Omega(m)}E_{q(V)}[v_n v_n^T] + \frac{\tau}{2}I\Big)^{-1}$$
$$\Big(\lambda\sum_{n\in\Omega(m)}(r_{mn}E_{q(V)}[v_n]^T - E_{b_m^{(u)}}[b_m^{(u)}]E_{q(V)}[v_n]^T - E_{b_n^{(v)}}[b_n^{(v)}]E_{q(V)}[v_n]^T)\Big)u_m - u_m^T u_m\Big)\Big)$$

By completing the squares we obtain,

$$q(U) = K''' \prod_{m=1}^{M} exp\Big( - \frac{1}{2}\Big(\lambda\sum_{n\in\Omega(m)}E_{q(V)}[v_n v_n^T] + \tau I\Big)$$
$$\Big(u_m - \Big(\lambda\sum_{n\in\Omega(m)}E_{q(V)}[v_n v_n^T] + \tau I\Big)^{-1}\Big(\lambda\sum_{n\in\Omega(m)}(r_{mn} - E_{b_m^{(u)}}[b_m^{(u)}] - E_{b_n^{(v)}}[b_n^{(v)}])E_{q(V)}[v_n]\Big)\Big)^T$$
$$\Big(u_m - \Big(\lambda\sum_{n\in\Omega(m)}E_{q(V)}[v_n v_n^T] + \tau I\Big)^{-1}\Big(\lambda\sum_{n\in\Omega(m)}(r_{mn} - E_{b_m^{(u)}}[b_m^{(u)}] - E_{b_n^{(v)}}[b_n^{(v)}])E_{q(V)}[v_n]\Big)\Big)$$
$$= \prod_{m=1}^{M} N\Big(u_m; \Big(\lambda\sum_{n\in\Omega(m)}E_{q(V)}[v_n v_n^T] + \tau I\Big)^{-1}\Big(\lambda\sum_{n\in\Omega(m)}(r_{mn} - E_{b_m^{(u)}}[b_m^{(u)}] - E_{b_n^{(v)}}[b_n^{(v)}])E_{q(V)}[v_n]\Big),$$

$$\Big(\lambda \sum_{n\in\Omega(m)} E_{q(V)}[v_n v_n^T] + \tau I\Big)^{-1}\Big)$$

$$q(U) = \prod_{m=1}^{M} \mathcal{N}\Big(u_m; \Big(\lambda \sum_{n\in\Omega(m)} (Var_{q(V)}(v_n) + E_{q(V)}[v_n]E_{q(V)}[v_n]^T) + \tau I\Big)^{-1}$$
$$\Big(\lambda \sum_{n\in\Omega(m)} (r_{mn} - E_{b_m^{(u)}}[b_m^{(u)}] - E_{b_n^{(v)}}[b_n^{(v)}])E_{q(V)}[v_n]\Big),$$
$$\Big(\lambda \sum_{n\in\Omega(m)} (Var_{q(V)}(v_n) + E_{q(V)}[v_n]E_{q(V)}[v_n]^T) + \tau I\Big)^{-1}\Big)$$

### A.4.3. User and Movie biases update

$$\frac{dL[q]}{dq(b^{(u)})} = E_{q(U)q(V)q(b^{(v)})}\Big[log\Big(P(R,U,V,b^{(u)},b^{(v)})\Big)\Big] - log\Big(q(b^{(u)})\Big) + C$$

$$\frac{dL[q]}{q(b^{(u)})} = 0, \quad log\left(q(U)\right) = E_{q(V)q(b^{(u)})q(b^{(v)})}\Big[log\Big(P(R,U,V,b^{(u)},b^{(v)})\Big)\Big] + C$$

$$q(b^{(u)}) = Kexp\Big(E_{q(U)q(V)q(b^{(v)})}\Big[log\Big(P(R,U,V,b^{(u)},b^{(v)})\Big)\Big]\Big)$$
$$= Kexp\Big(E_{q(U)q(V)q(b^{(v)})}\Big[log(P(R|U,V,b^{(u)},b^{(v)})$$
$$+ log(P(U)) + log(P(V)) + log(P(b^{(u)})) + log(P(b^{(v)}))$$

Absorbing the constant terms into $K$ gives us,

$$q(b^{(u)}) = K'exp\Big(E_{q(U)q(V)q(b^{(v)})}\Big[-\frac{\lambda}{2}\sum_{m=1}^{M}\sum_{n\in\Omega(m)}(r_{mn} - u_m^T v_n - b_m^{(u)} - b_n^{(v)})^2$$
$$-\frac{\tau}{2}\sum_{m=1}^{M}u_m^T u_m - \frac{\tau}{2}\sum_{n=1}^{N}v_n^T v_n - \frac{\gamma}{2}\sum_{m=1}^{M}(b_m^{(u)})^2 - \frac{\gamma}{2}\sum_{n=1}^{N}(b_n^{(v)})^2\Big]\Big)$$
$$= K'exp\Big(E_{q(U)q(V)q(b^{(v)})}\Big[-\frac{\lambda}{2}\sum_{m=1}^{M}\sum_{n\in\Omega(m)}(r_{mn}^2 - 2r_{mn}u_m^T v_n - 2r_{mn}b_m^{(u)} - 2r_{mn}b_n^{(v)}$$
$$+ (u_m^T v_n)^2 + 2b_m^{(u)}u_m^T v_n + 2b_n^{(v)}u_m^T v_n + 2b_m^{(u)}b_n^{(v)} + {b_m^{(u)}}^2 + {b_n^{(v)}}^2)$$
$$-\frac{\tau}{2}\sum_{m=1}^{M}u_m^T u_m - \frac{\tau}{2}\sum_{n=1}^{N}v_n^T v_n - \frac{\gamma}{2}\sum_{m=1}^{M}(b_m^{(u)})^2 - \frac{\gamma}{2}\sum_{n=1}^{N}(b_n^{(v)})^2\Big]\Big)$$
$$= K'exp\Big(-\frac{\lambda}{2}\sum_{m=1}^{M}\sum_{n\in\Omega(m)}(r_{mn}^2 - 2r_{mn}E_{q(U)}[u_m]^T E_{q(V)}[v_n] - 2r_{mn}b_m^{(u)} - 2r_{mn}E_{b_n^{(v)}}[b_n^{(v)}]$$
$$+ E_{q(U)q(V)}[(u_m^T v_n)^2] + 2b_m^{(u)}(E_{q(U)}[u_m]^T E_{q(V)}[v_n]) + 2E_{b_n^{(v)}}[b_n^{(v)}](E_{q(U)}[u_m]^T E_{q(V)}[v_n])$$
$$+ 2b_m^{(u)}E_{b_n^{(v)}}[b_n^{(v)}] + {b_m^{(u)}}^2 + E_{b_n^{(v)}}[{b_n^{(v)}}^2])$$
$$-\frac{\tau}{2}\sum_{m=1}^{M}E_{q(U)}[u_m^T u_m] - \frac{\tau}{2}\sum_{n=1}^{N}E_{q(V)}[v_n^T v_n] - \frac{\gamma}{2}\sum_{m=1}^{M}{b_m^{(u)}}^2 - \frac{\gamma}{2}\sum_{n=1}^{N}E_{b_n^{(v)}}[{b_n^{(v)}}^2]\Big)$$

Absorbing the terms not depending on $b^{(u)}$ into $K'$ gives us,

$$q(b^{(u)}) = K''exp\Big(-\frac{\lambda}{2}\sum_{m=1}^{M}\sum_{n\in\Omega(m)}(-2r_{mn}b_m^{(u)} + 2b_m^{(u)}(E_{q(U)}[u_m]^T E_{q(V)}[v_n])$$

$$+ 2b_m^{(u)} E_{b_n^{(v)}}[b_n^{(v)}] + {b_m^{(u)}}^2) - \frac{\gamma}{2} \sum_{m=1}^{M} {b_m^{(u)}}^2 \Big)$$

$$= K'' \prod_{m=1}^{M} exp\Big( - \frac{\lambda}{2} \sum_{n \in \Omega(m)} (-2 r_{mn} b_m^{(u)} + 2 b_m^{(u)} (E_{q(U)}[u_m]^T E_{q(V)}[v_n])$$

$$+ 2b_m^{(u)} E_{b_n^{(v)}}[b_n^{(v)}] + {b_m^{(u)}}^2) - \frac{\gamma}{2} {b_m^{(u)}}^2 \Big)$$

$$= K'' \prod_{m=1}^{M} exp\Big( \lambda \sum_{n \in \Omega(m)} (r_{mn} - (E_{q(U)}[u_m]^T E_{q(V)}[v_n])$$

$$- E_{b_n^{(v)}}[b_n^{(v)}]) b_m^{(u)} - (\frac{\gamma}{2} + \frac{\lambda}{2} |\Omega(m)|) {b_m^{(u)}}^2 \Big)$$

$$= K'' \prod_{m=1}^{M} exp\Big( (\frac{\gamma}{2} + \frac{\lambda}{2} |\Omega(m)|) \Big( (\frac{\gamma}{2} + \frac{\lambda}{2} |\Omega(m)|)^{-1} \Big( \lambda \sum_{n \in \Omega(m)} (r_{mn} - (E_{q(U)}[u_m]^T E_{q(V)}[v_n])$$

$$- E_{b_n^{(v)}}[b_n^{(v)}]) b_m^{(u)} \Big) - {b_m^{(u)}}^2 \Big) \Big)$$

By completing the squares we obtain,

$$q(b^{(u)}) = K''' \prod_{m=1}^{M} exp\Big( - \frac{1}{2} (\gamma + \lambda |\Omega(m)|) \Big( - (\gamma + \lambda |\Omega(m)|)^{-1} \Big( \lambda \sum_{n \in \Omega(m)} (r_{mn} - (E_{q(U)}[u_m]^T E_{q(V)}[v_n])$$

$$- E_{b_n^{(v)}}[b_n^{(v)}]) \Big) + b_m^{(u)} \Big)^2 \Big)$$

$$q(b^{(u)}) = \prod_{m=1}^{M} \mathcal{N}\Big( b_m^{(u)}; (\gamma + \lambda |\Omega(m)|)^{-1} \Big( \lambda \sum_{n \in \Omega(m)} (r_{mn} - (E_{q(U)}[u_m]^T E_{q(V)}[v_n]) - E_{b_n^{(v)}}[b_n^{(v)}]) \Big),$$

$$(\gamma + \lambda |\Omega(m)|)^{-1} \Big)$$

### A.4.4. EMBEDDINGS FACTORISATION

We can see that $u_m \sim N(u_m; \mu_m^{(u)}, \Sigma_m^{(u)})$ with

$$\Sigma_m^{(u)} = \Big( \lambda \sum_{n \in \Omega(m)} (Var_{q(V)}(v_n) + E_{q(V)}[v_n] E_{q(V)}[v_n]^T) + \tau I \Big)^{-1}$$

$$\mu_m^{(u)} = \Sigma_m^{(u)} \Big( \lambda \sum_{n \in \Omega(m)} (r_{mn} - E_{b_m^{(u)}}[b_m^{(u)}] - E_{b_n^{(v)}}[b_n^{(v)}]) E_{q(V)}[v_n] \Big)$$

We propose to further factorise the above expression, which gives us

$$q(u_m) \approx \prod_{k=1}^{K} q(u_{mk})$$

The KL divergence can be expressed as follows:

$$D_{KL}\Big( \prod_{k=1}^{K} q(u_{mk}) || q(u_m) \Big) = - \int \cdots \int \prod_{k=1}^{K} q(u_{mk}) log\Big( \frac{q(u_m)}{\prod_{k=1}^{K} q(u_{mk})} \Big) du_{m1} \dots du_{mK}$$

The functional derivative of the KL divergence in regard to the $l$-th component gives us:

$$\frac{d\, D_{KL}\Big( \prod_{k=1}^{K} q(u_{mk}) || q(u_m) \Big)}{d\, q(u_{ml})} = \frac{d\Big( - \int \cdots \int \prod_{k=1}^{K} q(u_{mk}) log\Big( \frac{q(u_m)}{\prod_{k=1}^{K} q(u_{mk})} \Big) du_{m1} \dots du_{mK} \Big)}{d\, q(u_{ml})}$$

Focusing only on the terms depending on $q(u_{ml})$, we get

$$\frac{d\, D_{KL}\Big(\prod_{k=1}^{K} q(u_{mk})||q(u_m)\Big)}{d\, q(u_{ml})} = \frac{d\Big(-\int q(u_{ml}) \underset{l\neq k}{E}[log(q(u_m))]du_{ml} + \int q(u_{ml})\log(q(u_{ml}))du_{ml} + C\Big)}{d\, q(u_{ml})}$$
$$= -\underset{l\neq k}{E}[log(q(u_m))] + log(q(u_{ml})) + 1$$

with

$$\underset{l\neq k}{E}[log(q(u_m))] = \int \cdots \int \Big(\prod_{l\neq k} q(u_{mk}) log(q(u_m)) du_{mk}\Big)$$

The optimal solution can be expressed as follows:

$$log(q(u_{ml})) = \underset{l\neq k}{E}[log(q(u_m))] + C \quad \text{with } C \text{ a constant insuring normalisation}$$
$$q(u_{ml}) = K \exp\Big(\underset{l\neq k}{E}[log(q(u_m))]\Big)$$

Absorbing the constant terms into $K$ gives us

$$q(u_{ml}) = K' \exp\Big(-\frac{1}{2}\underset{l\neq k}{E}[(u_m - \mu_m^{(u)})^T \Sigma_m^{(u)^{-1}}(u_m - \mu_m^{(u)})]\Big)$$

We denote $\Lambda = \Sigma^{-1}$, hence we can write

$$q(u_{ml}) = K' \exp\Big(-\frac{1}{2}\underset{l\neq k}{E}[(u_m - \mu_m^{(u)})^T \Lambda (u_m - \mu_m^{(u)})]\Big)$$
$$= K' \exp\Big(-\frac{1}{2}\underset{l\neq k}{E}\Big[\sum_{i}^{K}\sum_{j}^{K}(u_{mi} - \mu_{mi}^{(u)})\Lambda_{ij}(u_{mj} - \mu_{mj}^{(u)})\Big]\Big)$$

Considering that $\Lambda$ is symmetric, we can write

$$q(u_{ml}) = K' \exp\Big(-\frac{1}{2}\underset{l\neq k}{E}\Big[\sum_{i}^{K}(u_{mi} - \mu_{mi}^{(u)})\Lambda_{ii}(u_{mi} - \mu_{mi}^{(u)}) + 2\sum_{i}^{K}\sum_{i\leq j\leq K}(u_{mi} - \mu_{mi}^{(u)})\Lambda_{ij}(u_{mj} - \mu_{mj}^{(u)})\Big]\Big)$$

By absorbing the terms not depending on $u_{ml}$ into $K'$ we get

$$q(u_{ml}) = K'' \exp\Big(-\frac{1}{2}\underset{l\neq k}{E}\Big[(u_{ml} - \mu_{ml}^{(u)})^2\Lambda_{ll} + 2\sum_{i\neq l}^{K}(u_{ml} - \mu_{ml}^{(u)})(u_{mi} - \mu_{mi}^{(u)})\Lambda_{il}\Big]\Big)$$
$$= K'' \exp\Big(-\frac{\Lambda_{ll}}{2}\underset{l\neq k}{E}\Big[(u_{ml} - \mu_{ml}^{(u)})^2 + 2(u_{ml} - \mu_{ml}^{(u)})\Big(\sum_{i\neq l}^{K}(u_{mi} - \mu_{mi}^{(u)})\frac{\Lambda_{il}}{\Lambda_{ll}}\Big)\Big]\Big)$$

Given the linearity of the expectation, we have

$$q(u_{ml}) = K'' \exp\Big(-\frac{\Lambda_{ll}}{2}\Big((u_{ml} - \mu_{ml}^{(u)})^2 + 2(u_{ml} - \mu_{ml}^{(u)})\Big(\sum_{i\neq l}^{K}(\underset{l\neq k}{E}[u_{mi} - \mu_{mi}^{(u)}])\frac{\Lambda_{il}}{\Lambda_{ll}}\Big)\Big)\Big)$$
$$= K'' \exp\Big(-\frac{\Lambda_{ll}}{2}\Big((u_{ml} - \mu_{ml}^{(u)})^2 + 2(u_{ml} - \mu_{ml}^{(u)})\Big(\sum_{i\neq l}^{K}(E[u_{mi}] - \mu_{mi}^{(u)})\frac{\Lambda_{il}}{\Lambda_{ll}}\Big)\Big)\Big)$$
$$q(u_{ml}) = K'' \exp\Big(-\frac{\Lambda_{ll}}{2}(u_{ml} - \mu_{ml}^{(u)})^2\Big)$$
$$q(u_{ml}) = \mathcal{N}(u_{ml}; \mu_{ml}^{(u)}, \Lambda_{ll}^{-1})$$

### A.4.5. Full expression of $L[q]$

Given a full factorisation of the posterior distribution, and thus full independence, $L[q]$ can be expressed as follows:

$$
\begin{aligned}
L[q] &= E_{q(u)q(v)q(b^{(u)})q(b^{(v)})}[log\frac{P(U,V,b^{(u)},b^{(v)},R)}{q(U)q(V)q(b^{(u)})q(b^{(v)})}] \\
&= E_{q(u)q(v)q(b^{(u)})q(b^{(v)})}[log(P(U,V,b^{(u)},b^{(v)},R))] - E_{q(u)q(v)q(b^{(u)})q(b^{(v)})}[log(q(U))] \\
&\quad - E_{q(u)q(v)q(b^{(u)})q(b^{(v)})}[log(q(V))] - E_{q(u)q(v)q(b^{(u)})q(b^{(v)})}[log(q(b^{(u)}))] \\
&\quad - E_{q(u)q(v)q(b^{(u)})q(b^{(v)})}[log(q(b^{(v)}))] \\
&= E_{q(u)q(v)q(b^{(u)})q(b^{(v)})}[log(P(U,V,b^{(u)},b^{(v)},R))] \\
&\quad - E_{q(u)}[log(q(U))] - E_{q(v)}[log(q(V))] - E_{q(b^{(u)})}[log(q(b^{(u)}))] - E_{q(b^{(v)})}[log(q(b^{(v)}))] \\
&= E_{q(u)q(v)q(b^{(u)})q(b^{(v)})}[log(P(R|U,V,b^{(u)},b^{(v)}) + log(P(U)) + log(P(V)) + log(P(b^{(u)})) + log(P(b^{(v)}))] \\
&\quad - E_{q(u)}[log(q(U))] - E_{q(v)}[log(q(V))] - E_{q(b^{(u)})}[log(q(b^{(u)}))] - E_{q(b^{(v)})}[log(q(b^{(v)}))] \\
&= -\frac{\lambda}{2}\sum_{m=1}^{M}\sum_{n\in\Omega(m)}(r_{mn}^2 - 2r_{mn}E_{q(U)}[u_m]^T E_{q(V)}[v_n] - 2r_{mn}E_{b_m^{(u)}}[b_m^{(u)}] - 2r_{mn}E_{b_n^{(v)}}[b_n^{(v)}] \\
&\quad + E_{q(U)q(V)}[(u_m^T v_n)^2] + 2E_{b_m^{(u)}}[b_m^{(u)}](E_{q(U)}[u_m]^T E_{q(V)}[v_n]) + 2E_{b_n^{(v)}}[b_n^{(v)}](E_{q(U)}[u_m]^T E_{q(V)}[v_n]) \\
&\quad + 2E_{b_m^{(u)}}[b_m^{(u)}]E_{b_n^{(v)}}[b_n^{(v)}] + E_{b_m^{(u)}}[{b_m^{(u)}}^2] + E_{b_n^{(v)}}[{b_n^{(v)}}^2]) \\
&\quad - \frac{\tau}{2}\sum_{m=1}^{M}E_{q(U)}[u_m^T u_m] - \frac{\tau}{2}\sum_{n=1}^{N}E_{q(V)}[v_n^T v_n] - \frac{\gamma}{2}\sum_{m=1}^{M}E_{b_m^{(u)}}[{b_m^{(u)}}^2] - \frac{\gamma}{2}\sum_{n=1}^{N}E_{b_n^{(v)}}[{b_n^{(v)}}^2] \\
&\quad - \sum_{m=1}^{M}E_{q(u_m)}[log(q(u_m))] - \sum_{n=1}^{N}E_{q(v_n)}[log(q(v_n))] \\
&\quad - \sum_{m=1}^{M}E_{q(b_m^{(u)})}[log(q(b_m^{(u)}))] - \sum_{n=1}^{N}E_{q(b_n^{(v)})}[log(q(b_n^{(v)}))] \\
&\quad + C \\
L[q] &= -\frac{\lambda}{2}\sum_{m=1}^{M}\sum_{n\in\Omega(m)}(r_{mn}^2 - 2r_{mn}{\mu_m^{(u)}}^T\mu_n^{(v)} - 2r_{mn}\mu_m^{(bu)} - 2r_{mn}\mu_n^{(bv)} \\
&\quad + tr((\Sigma_m^{(u)} + \mu_m^{(u)}{\mu_m^{(u)}}^T)(\Sigma_n^{(v)} + \mu_n^{(v)}{\mu_n^{(v)}}^T)) + 2\mu_m^{(bu)}({\mu_m^{(u)}}^T\mu_n^{(v)}) + 2\mu_n^{(bv)}({\mu_m^{(u)}}^T\mu_n^{(v)}) \\
&\quad + 2\mu_m^{(bu)}\mu_n^{(bv)} + {\mu_m^{(bu)}}^2 + {\sigma_m^{(bu)}}^2 + {\mu_n^{(bv)}}^2 + {\sigma_n^{(bv)}}^2) \\
&\quad - \frac{\tau}{2}\sum_{m=1}^{M}({\mu_m^{(u)}}^T\mu_m^{(u)} + tr(\Sigma_m^{(u)})) - \frac{\tau}{2}\sum_{n=1}^{N}({\mu_n^{(v)}}^T\mu_n^{(v)} + tr(\Sigma_n^{(v)})) \\
&\quad - \frac{\gamma}{2}\sum_{m=1}^{M}({\mu_m^{(bu)}}^2 + {\sigma_m^{(bu)}}^2) - \frac{\gamma}{2}\sum_{n=1}^{N}({\mu_n^{(bv)}}^2 + {\sigma_n^{(bv)}}^2) \\
&\quad + \frac{1}{2}\sum_{m=1}^{M}log(|\Sigma_m^{(u)}|) + \frac{1}{2}\sum_{n=1}^{N}log(|\Sigma_n^{(v)}|) \\
&\quad + \frac{1}{2}\sum_{m=1}^{M}log({\sigma_m^{(bu)}}^2) + \frac{1}{2}\sum_{n=1}^{N}log({\sigma_n^{(bv)}}^2) \\
&\quad + C'
\end{aligned}
$$

### A.4.6. MAKING PREDICTION

Given the set of all rating $R$, the rating distribution of a user $m$ given to a movie $n$ is obtained through the following expression:

$$\begin{aligned}
p(r_{mn}|R) &= \int p(r_{mn}|u_m, v_n, b_m^{(u)}, b_n^{(v)}, R) \\
&\quad \times q(u_m)q(v_n)q(b_m^{(u)})q(b_n^{(u)})d\{u_m, v_n, b_m^{(u)}, b_n^{(v)}\} \\
&= \int \mathcal{N}(r_{mn}; u_m^T v_n + b_m^{(u)} + b_n^{(v)}, \lambda^{-1}) \\
&\quad \times q(u_m)q(v_n)q(b_m^{(u)})q(b_n^{(u)})d\{u_m, v_n, b_m^{(u)}, b_n^{(v)}\} \\
p(r_{mn}|R) &= E_{q(u)q(v)q(b^{(u)})q(b^{(v)})}[\mathcal{N}(r_{mn}; u_m^T v_n + b_m^{(u)} + b_n^{(v)}, \lambda^{-1})]
\end{aligned}$$

When predicting the rating given by a user $m$ to the movie $n$, we propose to use the expectation.

$$\begin{aligned}
E[r_{mn}|R] &= \int r p(r_{mn}|R) dr \\
&= \int r E_{q(u)q(v)q(b^{(u)})q(b^{(v)})}\Big[\mathcal{N}(r_{mn}; u_m^T v_n + b_m^{(u)} + b_n^{(v)}, \lambda^{-1})\Big] dr \\
&= \int E_{q(u)q(v)q(b^{(u)})q(b^{(v)})}\Big[r\mathcal{N}(r_{mn}; u_m^T v_n + b_m^{(u)} + b_n^{(v)}, \lambda^{-1})\Big] dr \\
&= E_{q(u)q(v)q(b^{(u)})q(b^{(v)})}\Big[\int r\mathcal{N}(r_{mn}; u_m^T v_n + b_m^{(u)} + b_n^{(v)}, \lambda^{-1}) dr\Big] \\
E[r_{mn}|R] &= E_{q(u)q(v)q(b^{(u)})q(b^{(v)})}\Big[u_m^T v_n + b_m^{(u)} + b_n^{(v)}\Big]
\end{aligned}$$

Due to the independance of $u_m$ and $v_n$, we can write

$$E[r_{mn}|R] = {\mu_m^{(u)}}^T \mu_n^{(v)} + \mu_m^{(bu)} + \mu_n^{(bv)}$$